

MicroInvest

A Multi-Investment Platform

Ji Chengyu Student ID G2608005K

SC6113 Individual Assignment: Financial Decentralized Application

7 October 2026

1 Introduction

MicroInvest is an educational financial DApp for investing beginners. Any connected user can deploy an independent Investment contract, give it a title and explanation, and publish it in a searchable directory. Other users discover a project, open its own detail page, deposit Sepolia test ETH, inspect their shares and redeem principal at any time. The application is available at [MicroInvest on Render](#).

Each pool retains simple rules: 1 ETH equals 1 internal share, including fractions; no yield, platform fee or share transfer is offered. The creator has no privilege to withdraw other participants' funds. The project demonstrates discovery, custody, ownership and confirmed transactions rather than a real asset strategy.

2 Problem Statement

A beginner needs to find the intended project and understand where funds are held, how contributions determine ownership and how principal can be recovered. A contract address alone is hard to recognize. A single unnamed pool also offers no convenient way to choose between independent projects.

MicroInvest associates a public title and explanation with a verified contract and a unique directory ID. Name/keyword search improves discovery; separate contract ledgers isolate participants' positions. Exact share conversion, clear transaction status and accessible history make the contribution and redemption process visible.

3 Objectives

The objectives are to provide user-created named pools and keyword discovery; integrate MetaMask with a responsive web interface; implement exact fractional shares and caller-bound redemption; persist project metadata and confirmed activity; and evaluate authorization, error handling, usability and gas. The submission follows the coursework requirements and all 13 report sections [1].

4 System Architecture

The browser uses Flask APIs for project search, metadata, pool state, positions and history. MetaMask signs deployments, deposits and redemptions directly on Sepolia. After deployment, the creator signs a separate registration message; Flask verifies the actual creation and recovered signer before publishing metadata. Flask holds no wallet private key.

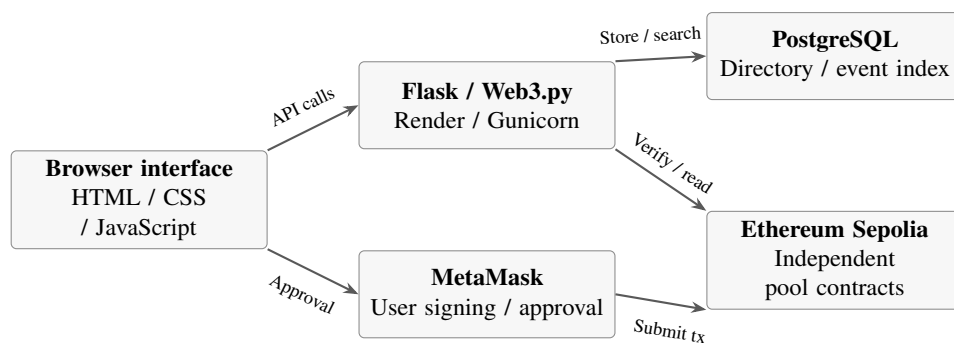


Figure 1: Metadata and confirmed history use a database; funds and entitlements remain on-chain.

PostgreSQL stores the directory and an index of confirmed events. Contract state remains authoritative for balances and redemption rights. Local SQLite is a development fallback. Every project-specific query selects its own contract and deployment block; losing directory data does not move funds, but prevents discovery until metadata is restored.

5 Technologies Used

Table 1: Technologies and responsibilities

Layer	Technology	Role
Contract	Solidity 0.8.30	Per-pool custody, exact shares and guarded redemption
Development	Node.js 24, Hardhat 3, pnpm	Compile, local chain, contract/browser tests
Frontend	HTML, CSS, JavaScript, ethers 6	Discovery, project pages and wallet requests
Backend	Python 3.12, Flask, Web3.py	Signed registration and validated chain reads
Database	SQLAlchemy, PostgreSQL / SQLite	Online metadata/event persistence; local fallback
Wallet / network	MetaMask, Ethereum Sepolia	User signing and testnet execution
Hosting / workflow	Render, Gunicorn, GitHub, VS Code	Hosted Python service and development

Node.js is development tooling; the hosted backend remains Python. Committed contract artifacts and the ethers bundle avoid a Node/Solidity build on Render. Gunicorn serves Flask, and DATABASE_URL connects SQLAlchemy to PostgreSQL [6]. SQLite files are unsuitable for Render’s ephemeral filesystem [5].

6 Smart Contract Design

Each Investment deploys the unchanged `MicroInvest.sol` bytecode with its own balance and ledger. `shares[address]` and `totalShares` use unsigned integers. One displayed share has 10^{18} units, so deposited wei equals credited share units without rounding.

Table 2: State transitions and contract validation

Method	Transition	Validation
<code>deposit()</code>	Credit caller and total by <code>msg.value</code>	Positive deposit
<code>withdraw(amountWei)</code>	Debit caller/total; send equal wei to caller	Positive amount within caller shares
<code>withdrawAll()</code>	Redeem caller’s complete position	Nonzero caller shares
Ledger / accounting reads	Return shares, principal, balance and excess	No state change

For example, a 0.003 ETH deposit creates 0.003 shares; a 0.001 ETH redemption leaves 0.002. Full exit clears that position. Gas is paid separately and is not reimbursed with principal. Each pool enforces:

$$\text{totalShares} = \sum_i \text{shares}_i, \quad \text{redeemableWei}_i = \text{shares}_i, \quad \text{balanceWei} \geq \text{totalShares}.$$

Authorization follows `msg.sender`. A creator can participate but cannot redeem anyone else’s position. There is no owner, pause, upgrade, transferable token or investment manager. Pool A and Pool B have separate storage even when the same wallet participates in both.

Withdrawals validate entitlement, debit state and then transfer ETH. A reentrancy lock blocks nested calls; failed transfers revert the ledger change. These decisions follow checks-effects-interactions [2]. Adversarial receivers test refusal of ETH and attempted nested withdrawal.

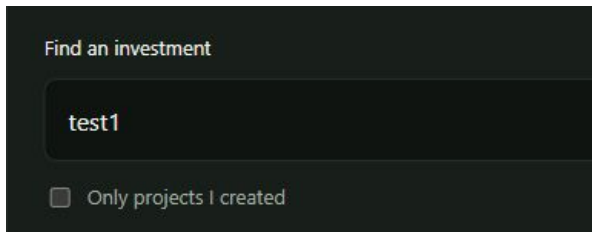
Deposited and Withdrawn events identify the investor, amount and resulting shares. Only successful confirmed events enter business history. Pending/reverted transactions and wallet rejections create no successful event row. Forced ETH is excess balance, not new shares or yield.

The optimizer uses 200 runs; operations change a fixed amount of storage without iterating over investors. Historical receipt gas appears in Section 9. Reusing tested bytecode adds independent projects without adding creator privileges or changing principal accounting.

7 Application Design

The English interface begins with an Explore directory. Users search names or words from explanations, browse paged results, or filter projects by the connected creator. Matching is literal and case-insensitive; whitespace-separated terms must all match. IDs and addresses distinguish projects even when titles are identical. This is keyword search, not semantic search.

A project opens `/investments/<id>` with its title, explanation, contract address, pool metrics, deposit/redemption controls and personal position. Its own activity page preserves project and wallet scope. The detail preview shows the five newest confirmed records; full activity loads 20-row pages and older records. Empty recent block ranges are traversed automatically.



The live filter uses `test1`. The matching project card identifies Investment #2, its explanation and the contract link. The complete capture is retained with the report source.

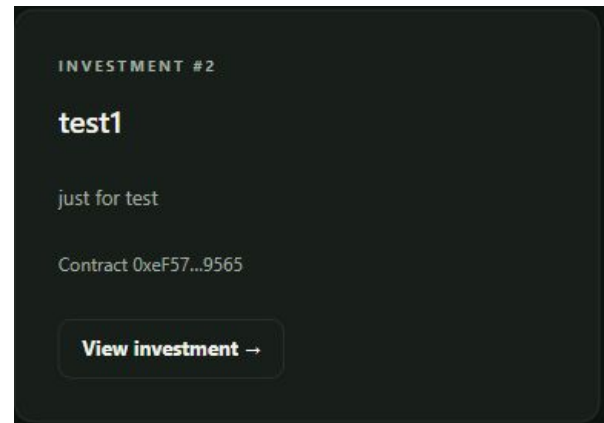


Figure 2: Two readable crops from one live search capture: the keyword filter and its matching independently deployed project.

The wallet menu lists only accounts authorized by MetaMask. Selecting one refreshes its positions/history; every transaction explicitly uses that selected signer. Provider account/network events follow EIP-1193 [4]. Wrong-network or unavailable-data states disable writes. Account changes cancel stale responses and do not carry another account's pending state into the current view.

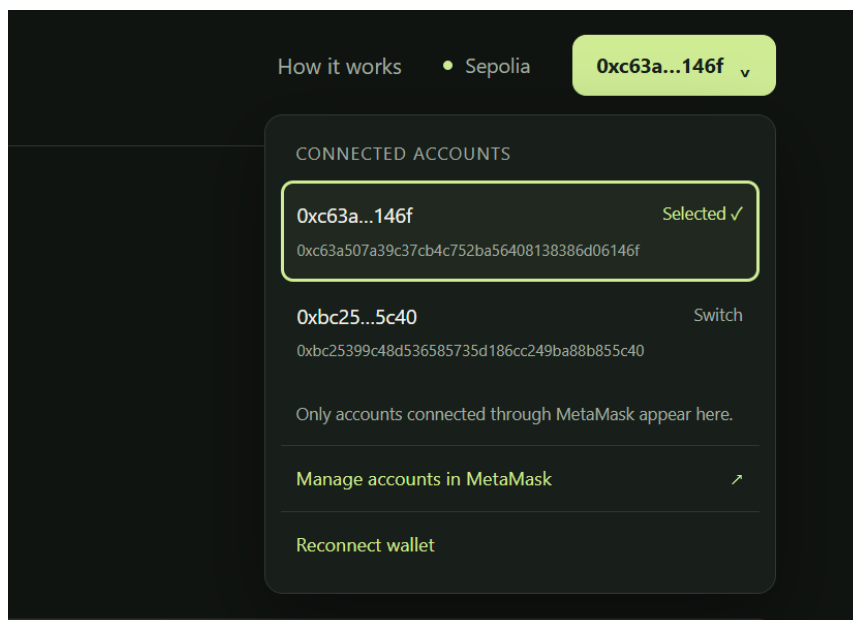


Figure 3: Student-supplied MetaMask account menu on Sepolia. The same account-selection behavior is retained on the updated project pages.

Approval, pending hash, mined success and revert are distinct states. On confirmation the page rereads the selected contract; delayed confirmation retains the hash for recovery. Exact input validation uses integer wei and BigInt. The UI explains test ETH, fixed shares, no returns/platform fee and separate network gas.

8 Implementation

The backend separates `catalog.py` (metadata/search), `chain.py` (validated blockchain reads) and `history.py` (persistent activity synchronization). Templates and browser modules provide Explore, creation, detail and full-history pages. Jinja escaping and `textContent` keep creator text as data.

Creation has two approvals: MetaMask first deploys the supported artifact; after a successful receipt the creator signs an EIP-191 registration message [3]. The message binds the title, explanation, creation hash, chain and HTTP origin. Flask checks successful direct creation, exact creation/runtime bytecode and signer equality with the actual deployer. It saves server-derived address/block/creator and returns a unique ID. Identical retries reuse the same entry; conflicting metadata is rejected. A declined registration can resume with the existing creation hash, avoiding a second deployment.

Figure 4: Updated creation form with required title and explanation. This capture illustrates the form without submitting a deployment.

Table 3: Database responsibilities and identity keys

Table	Stored data / identity
<code>investments</code>	ID, network, unique address/creation hash, creator, title, explanation, deployment block, search text and UTC creation time
<code>history_events</code>	Wallet, exact wei/shares, block/hash/time; unique network + contract + transaction hash + log index
<code>history_ranges</code>	Network, contract, wallet, completed start/end blocks and canonical end-block hash

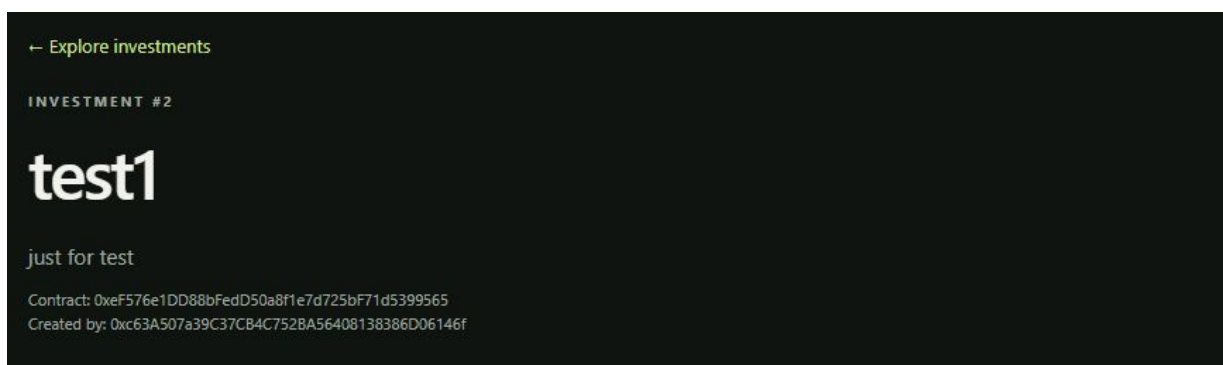


Figure 5: Live independent detail page for directory ID 2, `test1`, linked to its own Sepolia contract.

API requests carry `investment=<id>`; invalid IDs fail instead of falling back to another pool. SQL parameters constrain search. The published schema is `docs/schema.sql`; `scripts/export-catalog.py--include-history` exports public directory/index data. The database holds no private keys and cannot authorize a withdrawal.

9 Testing and Results

Automated evaluation uses contract tests, backend tests, frontend tests and a real browser against Flask and a local Hardhat chain. The automated wallet simulates EIP-1193; backend unit tests mock RPC. Real MetaMask/Render testing is a separate student-reported acceptance result.

Table 4: Recorded automated results for the updated source

Suite	Result	Coverage
Contract	12 passed	Unchanged contract; accounting, caller isolation, rollback/reentrancy, 40-operation sequence
Backend	54 passed	Metadata/signatures, search, project scope, persistent history, checkpoints and errors
Frontend	9 passed	Exact amounts/errors, empty-range paging, cancellation and cursor progress
Browser	24 checks passed	Creation/retry, two creators/pools, deposits/exit, scope, mobile layout and restart recovery
PostgreSQL integration	1 skipped locally	Dedicated PostgreSQL case configured in CI; not claimed as a recorded pass

The local integration flow invests 0.005 ETH in one pool and 0.002 in another, then redeems the first while preserving the second. It checks keyword/creator search, immutable retry, account selection and catalog persistence after Flask restart. After 6,000 inactive blocks, a fresh detail page still recovers five newest records; full history loads all 21 local events without duplicates. Checkpoint changes rebuild the scoped index, while failed RPC synchronization preserves stored rows.

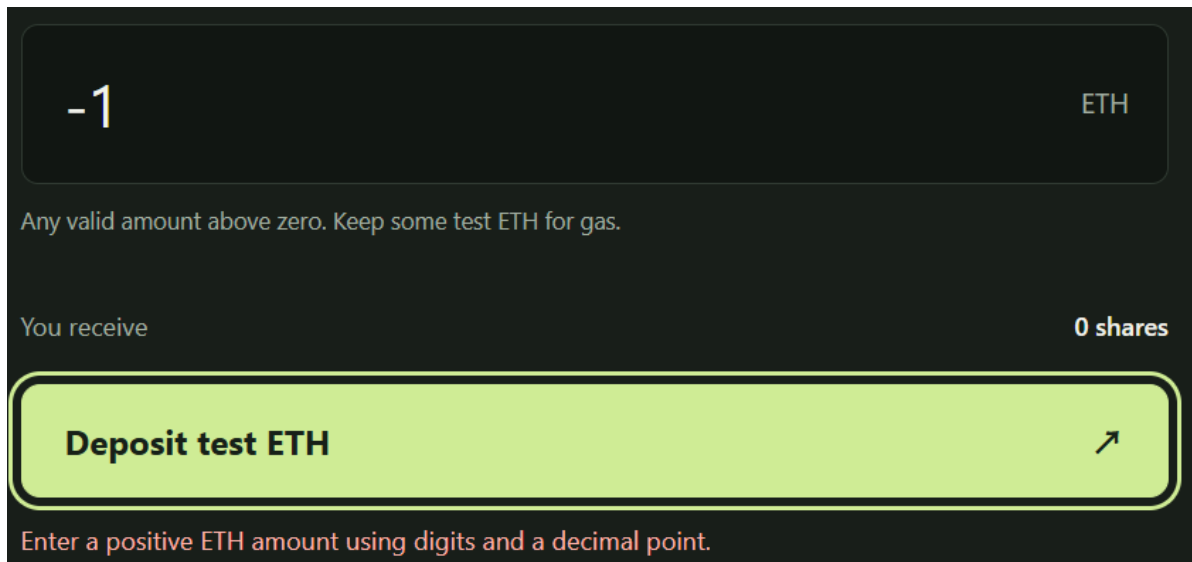


Figure 6: Student-supplied negative deposit validation (crop): -1 is rejected before a wallet transaction request.

Zero, nonnumeric, exponential and over-precision amounts are rejected. Excess redemption cannot spend another wallet's shares. Invalid/tampered registration signatures and unsupported deployment code fail. RPC/database failures produce explicit errors; unavailable positions are not invented as zero balances. A 390-pixel local viewport was checked for horizontal overflow.

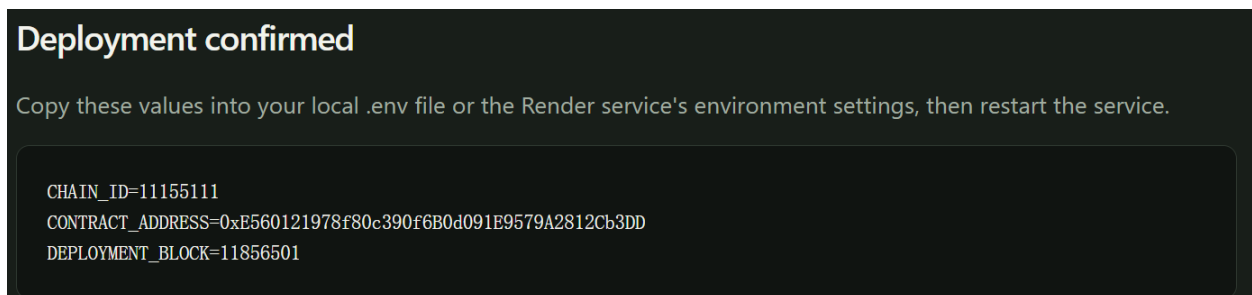


Figure 7: Original successful Sepolia deployment (crop), retained as a valid instance of the unchanged contract: chain 11155111, block 11856501.

Testing and Results (continued)

Live contracts and principal lifecycle

I successfully tested the updated application on Render with MetaMask after redeployment on 7 October 2026. Independent read-only checks verified the live directory/search, both pool APIs, the new browser module and database-backed history. The directory contains the original pool and a separately created test1 project:

Table 5: Observed live Investment contracts

ID	Title / creation block	Contract address
1	Original MicroInvest Pool / 11856501	0xE560121978f80c390f6B0d091E9579A2812Cb3DD
2	test1 / 11863263	0xE576e1DD88bFedD50a8f1e7d725bF71d5399565

Both observed directory entries were created by the same participant; the two-creator isolation claim comes from the local browser suite. Earlier Sepolia transactions remain valid evidence of the unchanged contract's principal lifecycle:

Table 6: Earlier confirmed Sepolia operations and measured gas

Operation	ETH	Shares after	Block	Gas used
Deposit	0.003	0.003	11856700	245,994
Partial redemption	0.001	0.002	11856703	53,700
Full redemption	0.002	0	11856705	50,912

The original creation used 1,814,581 gas. Separate local deposit/partial/full-exit observations used 71,884/50,282/50,272 gas with different amounts and environments; these are not a controlled performance comparison. Actual fee is gas used times effective gas price. No new-project throughput or PostgreSQL latency benchmark was performed.

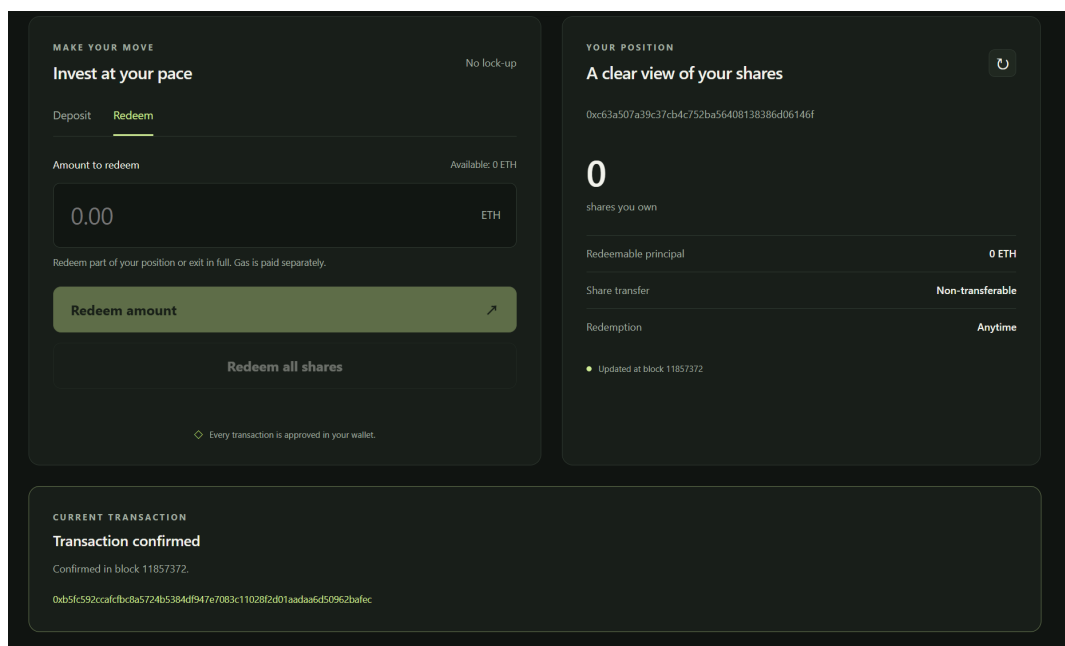


Figure 8: Student-supplied confirmed 0.04 ETH redemption at block 11857372, leaving zero personal shares and redeemable principal. This same transaction was recovered from the hosted history index.

[Original creation receipt](#) | [test1 creation transaction](#) | [Recovered redemption transaction](#)

Testing and Results (continued)

Database history and hosted deployment

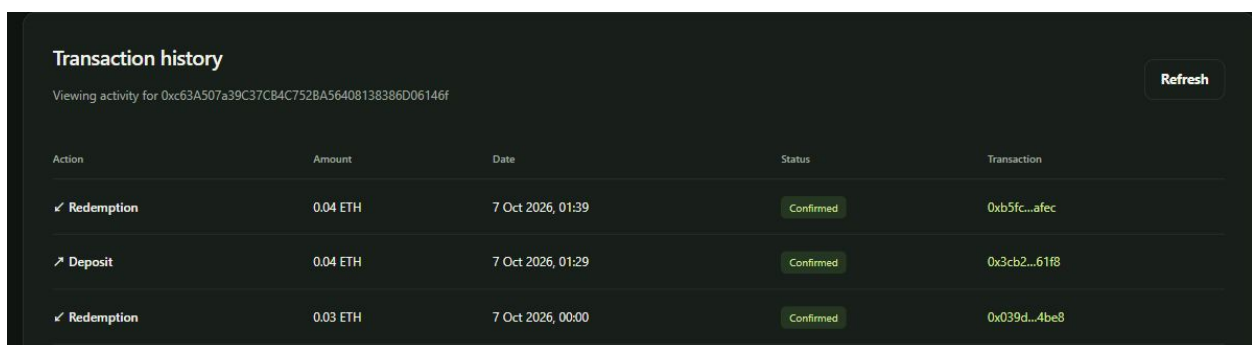
The history API now returns source: "database" and the latest five original-pool records, including the confirmed 0.04 ETH redemption in block 11857372. Completed scan ranges include empty intervals; missing ranges are fetched in windows of at most 5,000 blocks, using RPC chunks of at most 1,000. Exact events are upserted without duplicates, and subsequent queries use the stored coverage plus any newly mined blocks.

The server checks the latest stored end-block hash against the chain. A changed checkpoint invalidates only that wallet/pool index for rebuilding; a hash change during scanning aborts the write. The index improves recovery across application restarts but still requires RPC validation. The UI fills its five-record preview or 20-row history page across empty ranges rather than declaring an empty first range to be the complete history.

Verified live history response (abbreviated)

```
{
  "items": [{
    "type": "withdrawal", "amountEth": "0.04",
    "blockNumber": 11857372, "sharesAfter": "0",
    "transactionHash": "0xb5fc...afec"
  }, ...],
  "source": "database"
}
```

Figure 9: Selected fields from the independently verified live API response. The complete JSON is retained in the updated evidence file; this is an excerpt, not a browser screenshot.



The screenshot shows a 'Transaction history' page with a 'Refresh' button. Below the title, it says 'Viewing activity for 0xc63A507a39C37CB4C752BA56408138386D06146f'. The table has five columns: Action, Amount, Date, Status, and Transaction. It lists three transactions: a Redemption of 0.04 ETH, a Deposit of 0.04 ETH, and another Redemption of 0.03 ETH, all confirmed on 7 Oct 2026.

Action	Amount	Date	Status	Transaction
✓ Redemption	0.04 ETH	7 Oct 2026, 01:39	Confirmed	0xb5fc...afec
↗ Deposit	0.04 ETH	7 Oct 2026, 01:29	Confirmed	0x3cb2...61f8
✓ Redemption	0.03 ETH	7 Oct 2026, 00:00	Confirmed	0x039d...4be8

Figure 10: Current full-activity page for the original Investment, showing its recovered confirmed records in project-specific history.

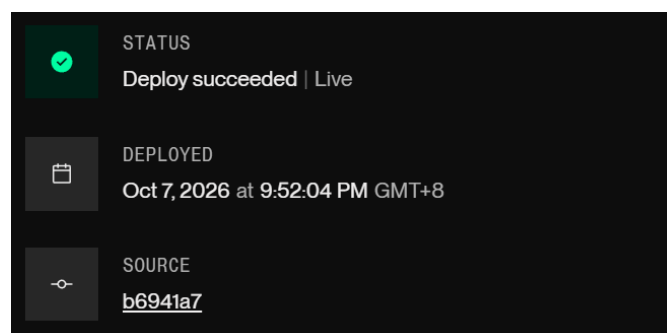


Figure 11: Student-supplied Render status showing the updated commit b6941a7 as Deploy succeeded / Live. The image is cropped to the status and source fields.

Render initializes the catalog before Gunicorn starts. After clearing the build cache and redeploying, public source files matched b6941a7 and database history succeeded. The earlier stale-code cause was not established. Process liveness at /healthz was checked separately from directory, pool and history reads.

Student acceptance, earlier receipts, local tests and current API checks are distinct evidence. Hosted restart persistence and the dedicated PostgreSQL test were not independently rerun in this revision.

10 Challenges Encountered

Exact integer wei and string API amounts avoided rounding across Python, Solidity and JavaScript. Selecting the authorized signer explicitly fixed reliance on the first wallet account. Adding projects required every request, position, history page and pending hash to carry its own contract/wallet context. Registration needed deployment verification and signature binding so a participant could not claim another creator's contract.

Old transactions appeared missing when the latest 5,000-block window was empty. Following the next cursor recovered them; persistent event/range tables and automatic older-range loading resolved the presentation issue. Separating live deployment status from actual endpoint verification also exposed stale served code during the hosted update.

11 Limitations

Pools hold principal without market allocation, diversification, yield or a guarantee of investment value. One mined confirmation is sufficient for interface feedback, but is not finality. Public RPC and database availability affect reads/discovery; initial historical backfill may require several requests. Literal search is not semantic search, and published titles/explanations cannot be edited or moderated in this version.

Addresses, contract activity and creator text are public; no identity profile or private key is stored. Signed metadata proves the deployer's authorization, not the truth of an investment explanation. Direct MetaMask deployments are supported; factory/smart-contract-wallet creation is outside scope. There is no external security audit, load test, novice-user study, complete accessibility audit or administrative rescue/upgrade path. The prototype is evaluated with test ETH rather than real savings. Render Free PostgreSQL expires 30 days after creation unless upgraded [5].

12 Future Improvements

Future work includes beginner usability studies, moderation and metadata amendment with clear authorization, better search relevance, multiple wallet providers and monitored incremental indexing. Database backups/restore exercises, hosted restart checks and broader reorganization testing would strengthen operational evidence. Any real strategy, transferable token or additional asset requires a separate financial specification and security review.

13 Conclusion

MicroInvest now combines independently created named contracts, keyword discovery and project-specific participation with the original exact-share/principal-exit rules. Creator-authorized registration and PostgreSQL retain the directory; confirmed activity is indexed while entitlements remain on-chain. Local tests cover accounting, authorization and isolation, and the updated Render APIs demonstrate live metadata and recovered history. The system meets the coursework's financial DApp learning objectives within its stated educational scope.

References

- [1] SC6113 Individual Assignment: Financial Decentralized Application. Supplied Word document, Sections III and VI–XI.
- [2] Solidity 0.8.30. *Security Considerations*. <https://docs.soliditylang.org/en/v0.8.30/security-considerations.html>.
- [3] Ethereum Improvement Proposals. *ERC-191: Signed Data Standard*. <https://eips.ethereum.org/EIPS/eip-191>.
- [4] Ethereum Improvement Proposals. *EIP-1193: Ethereum Provider JavaScript API*. <https://eips.ethereum.org/EIPS/eip-1193>.
- [5] Render. *Deploy a Flask App; Deploy for Free*. <https://render.com/docs/deploy-flask>; <https://render.com/docs/free>.
- [6] SQLAlchemy 2.0. *Table Configuration; PostgreSQL dialect*. <https://docs.sqlalchemy.org/en/20/core/metadata.html>; <https://docs.sqlalchemy.org/en/20/dialects/postgresql.html>.